Kafka Lab Record: Week 11 and Week 12

Kafka Streams Word Count and Kafka Integration with the Hadoop Ecosystem (Docker, Confluent Platform 7.4.0)

Week 11: Word Count Stream Processing with Kafka Streams

Aim

To implement a simple word count stream processing application using Kafka Streams.

Introduction

Kafka Streams is a Java library for building real-time applications that read data from Kafka topics, process it, and write results back to Kafka topics. It runs inside your own application and needs no separate processing cluster.

- KStream: an unbounded stream of records.
- KTable: a table of the latest value for each key. A count is stored in a KTable and updated as new records arrive.
- Topology: the chain of processing steps.

Topology used in this experiment:

```
streams-plaintext-input  ->  split lines into words  ->  group by word  ->
count  ->  streams-wordcount-output
```

Software Requirements

- Docker Desktop with Docker Compose
- JDK 11 or higher and Apache Maven 3.6 or higher
- Git Bash (terminal)
- Images: confluentinc/cp-zookeeper:7.4.0 and confluentinc/cp-kafka:7.4.0

Project Structure

```
Week-11/
├── docker-compose.yml
├── pom.xml
└── src/
    └── main/
        └── java/
            └── com/
                └── lab/
                    └── WordCountApp.java
```

Procedure

Step 1: Stop the containers of any earlier week (to free the ports), then create the project folders.

```
docker-compose down
mkdir -p Week-11/src/main/java/com/lab
cd Week-11
```

Step 2: Create `docker-compose.yml` (same as Week 9, with a fixed container name `kafka`).

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
```

```
      - 9093:9093
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Step 3: Start Kafka and confirm that the containers are running.

```
docker-compose up -d
docker ps
```

Step 4: Create the input and output topics.

```
docker exec kafka kafka-topics --create --topic streams-plaintext-input --
partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
docker exec kafka kafka-topics --create --topic streams-wordcount-output --
partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
```

Output:

```
Created topic streams-plaintext-input.
Created topic streams-wordcount-output.
```

Step 5: Create `pom.xml` with the Kafka Streams dependency.

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.lab</groupId>
  <artifactId>wordcount</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
```

```
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-streams</artifactId>
      <version>3.4.0</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>1.7.36</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <mainClass>com.lab.WordCountApp</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 6: Create `src/main/java/com/lab/WordCountApp.java`.

```
package com.lab;

import java.util.Arrays;
import java.util.Properties;
import java.util.concurrent.CountDownLatch;

import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.KTable;
import org.apache.kafka.streams.kstream.Produced;

public class WordCountApp {
```

```
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "wordcount-app");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9093");
        props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
Serdes.String().getClass());
        props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
Serdes.String().getClass());
        // Send every update immediately so the output is easy to observe
        props.put(StreamsConfig.COMMIT_INTERVAL_MS_CONFIG, 1000);
        props.put("statestore.cache.max.bytes", 0);

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, String> textLines = builder.stream("streams-
plaintext-input");

        KTable<String, Long> wordCounts = textLines
                .flatMapValues(line ->
Arrays.asList(line.toLowerCase().split("\\W+")))
                .filter((key, word) -> !word.isEmpty())
                .groupBy((key, word) -> word)
                .count();

        wordCounts.toStream()
                .to("streams-wordcount-output",
Produced.with(Serdes.String(), Serdes.Long()));

        KafkaStreams streams = new KafkaStreams(builder.build(), props);
        CountDownLatch latch = new CountDownLatch(1);

        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            streams.close();
            latch.countDown();
        }));

        streams.start();
        try {
            latch.await();
        } catch (InterruptedException e) {
```

```
            Thread.currentThread().interrupt();
        }
    }
}
```

Step 7 (Terminal 1): Build and run the application. It keeps running until you press Ctrl+C.

```
mvn clean compile exec:java
```

Output (last lines):

```
[main] INFO org.apache.kafka.streams.KafkaStreams - stream-client [wordcount-app-...] State transition from REBALANCING to RUNNING
```

Step 8 (Terminal 2): Start the console consumer on the output topic. The keys are words and the values are counts.

```
docker exec kafka kafka-console-consumer --topic streams-wordcount-output --from-beginning --bootstrap-server localhost:9092 --property print.key=true --property key.separator=" : " --property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

Step 9 (Terminal 3): Send three lines of text to the input topic.

```
printf 'hello kafka streams\nhello world\nkafka streams word count\n' | docker exec -i kafka kafka-console-producer --topic streams-plaintext-input --bootstrap-server localhost:9092
```

Step 10: Observe Terminal 2. Each time a word appears again, its count is updated and a new record is written.

```
hello : 1
kafka : 1
streams : 1
hello : 2
world : 1
kafka : 2
streams : 2
word : 1
count : 1
```

Step 11: Stop the application with Ctrl+C in Terminal 1 and the consumer with Ctrl+C in Terminal 2.

Result

The Kafka Streams application read lines from `streams-plaintext-input`, split them into words, counted each word, and wrote the running counts to `streams-wordcount-output`. The final counts were: hello 2, kafka 2, streams 2, world 1, word 1, count 1.

Week 12: Kafka Integration with the Hadoop Ecosystem

Aim

To implement Kafka integration with the Hadoop ecosystem by streaming data from a Kafka topic into HDFS.

Introduction

Kafka is used to collect real-time data, and Hadoop (HDFS) is used to store large volumes of data for batch analytics. The two are commonly integrated in these ways:

- Kafka Connect HDFS Sink connector: copies topic data into HDFS files (used in this experiment).
- Spark Streaming or Structured Streaming: reads from Kafka and processes or writes to HDFS and Hive.
- Apache NiFi or Flume: moves data between Kafka and Hadoop components.
- Hive and Spark SQL: query the files stored in HDFS.

Data flow in this experiment:

```
Console Producer  ->  Kafka topic hdfs-topic  ->  Kafka Connect (HDFS Sink)
->  HDFS (NameNode + DataNode)
```

Software Requirements

- Docker Desktop with Docker Compose (allocate at least 6 GB memory to Docker)
- Internet access (the HDFS connector is downloaded from Confluent Hub on first start)
- Images: cp-zookeeper, cp-kafka and cp-kafka-connect (7.4.0), bde2020/hadoop-namenode and bde2020/hadoop-datanode (2.0.0-hadoop3.2.1-java8)

Project Structure

```
Week-12/
├── docker-compose.yml
├── hadoop.env
└── hdfs-sink.json
```

Procedure

Step 1: Stop the Week 11 containers, then create the project folder.

```
docker-compose down
mkdir Week-12
cd Week-12
```

Step 2: Create `hadoop.env`, the common Hadoop settings.

```
CORE_CONF_fs_defaultFS=hdfs://namenode:9000
CORE_CONF_hadoop_http_staticuser_user=root
CORE_CONF_io_compression_codecs=org.apache.hadoop.io.compress.SnappyCodec

HDFS_CONF_dfs_webhdfs_enabled=true
HDFS_CONF_dfs_permissions_enabled=false
HDFS_CONF_dfs_namenode_datanode_registration_ip___hostname___check=false
```

Step 3: Create `docker-compose.yml` with ZooKeeper, Kafka, Hadoop (NameNode and DataNode) and Kafka Connect. The Connect container installs the HDFS connector when it starts.

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    container_name: kafka
    depends_on:
```

```
      - zookeeper
    ports:
      - 9092:9092
      - 9093:9093
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    environment:
      - CLUSTER_NAME=lab-cluster
    env_file:
      - ./hadoop.env
    ports:
      - 9870:9870
      - 9000:9000
    volumes:
      - namenode-data:/hadoop/dfs/name

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    depends_on:
      - namenode
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
    env_file:
      - ./hadoop.env
    volumes:
      - datanode-data:/hadoop/dfs/data

  kafka-connect:
    image: confluentinc/cp-kafka-connect:7.4.0
    container_name: kafka-connect
    depends_on:
```

```
      - kafka
      - namenode
    ports:
      - 8083:8083
    environment:
      CONNECT_BOOTSTRAP_SERVERS: kafka:9092
      CONNECT_REST_ADVERTISED_HOST_NAME: kafka-connect
      CONNECT_REST_PORT: 8083
      CONNECT_GROUP_ID: connect-cluster
      CONNECT_CONFIG_STORAGE_TOPIC: connect-configs
      CONNECT_OFFSET_STORAGE_TOPIC: connect-offsets
      CONNECT_STATUS_STORAGE_TOPIC: connect-status
      CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_KEY_CONVERTER: org.apache.kafka.connect.storage.StringConverter
      CONNECT_VALUE_CONVERTER:
org.apache.kafka.connect.storage.StringConverter
      CONNECT_PLUGIN_PATH: /usr/share/java,/usr/share/confluent-hub-
components
    command:
      - bash
      - -c
      - |
        confluent-hub install --no-prompt confluentinc/kafka-connect-
hdfs:latest
        /etc/confluent/docker/run

volumes:
  namenode-data:
  datanode-data:
```

Step 4: Start all services and check that five containers are running.

```
docker-compose up -d
docker ps
```

Step 5: Verify that HDFS is running with one live DataNode. The NameNode web page at http://localhost:9870 shows the same information.

```
docker exec namenode hdfs dfsadmin -report | grep "Live datanodes"
```

Output:

```
Live datanodes (1):
```

Step 6: Wait 1 to 2 minutes for Kafka Connect to install the connector and start, then check that the HDFS connector is available.

```
curl -s http://localhost:8083/connector-plugins | grep -o
"io.confluent.connect.hdfs.HdfsSinkConnector"
```

Output:

```
io.confluent.connect.hdfs.HdfsSinkConnector
```

Step 7: Create the Kafka topic that will be copied to HDFS.

```
docker exec kafka kafka-topics --create --topic hdfs-topic --partitions 1 --
replication-factor 1 --bootstrap-server localhost:9092
```

Output:

```
Created topic hdfs-topic.
```

Step 8: Create the connector configuration `hdfs-sink.json`. The connector writes a file to HDFS after every 3 records (`flush.size`).

```
{
  "name": "hdfs-sink",
  "config": {
    "connector.class": "io.confluent.connect.hdfs.HdfsSinkConnector",
    "tasks.max": "1",
    "topics": "hdfs-topic",
    "hdfs.url": "hdfs://namenode:9000",
    "flush.size": "3",
    "format.class": "io.confluent.connect.hdfs.json.JsonFormat",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false",
    "topics.dir": "/topics",
    "logs.dir": "/logs",
    "confluent.topic.bootstrap.servers": "kafka:9092",
    "confluent.topic.replication.factor": "1"
  }
}
```

Step 9: Register the connector and check its status. The connector and its task should be RUNNING.

```
curl -X POST -H "Content-Type: application/json" --data @hdfs-sink.json
http://localhost:8083/connectors
curl http://localhost:8083/connectors/hdfs-sink/status
```

Output:

```
{"name":"hdfs-sink","connector":{"state":"RUNNING","worker_id":"kafka-
connect:8083"},"tasks":[{"id":0,"state":"RUNNING","worker_id":"kafka-
connect:8083"}],"type":"sink"}
```

Step 10: Send three JSON records to the Kafka topic.

```
printf
'{"id":1,"name":"Ravi"}\n{"id":2,"name":"Anitha"}\n{"id":3,"name":"Kiran"}\n'
| docker exec -i kafka kafka-console-producer --topic hdfs-topic --bootstrap-
server localhost:9092
```

Step 11: Wait about 10 seconds, then list the files that the connector created in HDFS.

```
docker exec namenode hdfs dfs -ls -R /topics
```

Output (the connector may also create a +tmp folder):

```
drwxr-xr-x   - appuser supergroup   ...   /topics/hdfs-topic
drwxr-xr-x   - appuser supergroup   ...   /topics/hdfs-topic/partition=0
-rw-r--r--   3 appuser supergroup   ...   /topics/hdfs-
topic/partition=0/hdfs-topic+0+0000000000+0000000002.json
```

Step 12: Read the file from HDFS to confirm that the Kafka data is stored there.

```
docker exec namenode hdfs dfs -cat /topics/hdfs-topic/partition=0/hdfs-
topic+0+0000000000+0000000002.json
```

Output:

```
{"id":1,"name":"Ravi"}
{"id":2,"name":"Anitha"}
{"id":3,"name":"Kiran"}
```

Step 13: Delete the connector and stop all containers.

```
curl -X DELETE http://localhost:8083/connectors/hdfs-sink
docker-compose down
```

Result

A Hadoop cluster (NameNode and DataNode) was started with Docker, and the Kafka Connect HDFS Sink connector was configured. The three JSON records sent to the Kafka topic `hdfs-topic` were written automatically to a file in HDFS under `/topics/hdfs-topic/partition=0/`. This shows how Kafka can feed real-time data into the Hadoop ecosystem for storage and analysis.

Conclusion

Week 11 showed stateful stream processing with Kafka Streams, where word counts are updated continuously as data arrives. Week 12 showed Kafka integration with Hadoop, where Kafka Connect moves data from a topic into HDFS without custom code.